

İSTANBUL BİLGİ UNIVERSITY
FACULTY of
SCIENCE AND LETTERS

Quantum Computation : Implementation of Quantum Gates on Linux Clusters

SUBMITTED By
Arda Karaduman

In Partial Fulfillment of the Requirements for the Degree of
Bachelor of Science in Computer Science

July 25, 2004

Chris Stephenson
Head of Department

APPROVED By

Bülent Özel
Dissertation Supervisor

Statement of Originality

I, the undersigned, declare that this project is my own original work, and I give permission that it may be photocopied and made available for interlibrary loan.

Name:

Signed:

Date:

Abstract

This thesis gives a brief introduction to Quantum Computation and discusses implementation schemas for a Quantum Computer simulator on a distributed system. Research into Quantum Computers have increased drastically since Peter Shor showed that some problems which can only be solved in exponential time with classical computers can be solved in polynomial time with quantum computers. Other algorithms have also been discovered where Quantum Computers have an advantage over their classical counterparts. There are still great difficulties in physical realization of Quantum Computers, and so far machines with only a few bits of memory have been accomplished. Because of this, much of the research into Quantum Computation algorithms make use of simulators. Due to the fundamental differences between Classical Computers and Quantum Computers it is quite inefficient to simulate a Quantum Computer with a reasonable memory. This thesis investigates the possible strategies for simulating a Quantum Computer on a parallel Linux cluster.

Contents

Abstract	i
Table of Contents	ii
List of Figures	iii
1 Introduction	1
2 History of Quantum Computation	2
3 Models of Computation	4
4 Physical Background	5
5 Mathematical Basics	7
6 Quantum Computation	10
7 Simulation	13
8 Parallel Simulation	14
9 Library Documentation	16
10 Conclusion	21
References	22

List of Figures

- 1 Dr. Isaac Chuang, research staff member at IBM's Almaden Research Center (San Jose, Calif.), holds a quantum computer – a glass tube containing specially designed molecules that can solve some of the most difficult mathematical problems exponentially faster than a conventional computer. 12

1 Introduction

Since the first computers, the trend has been to miniaturize these machines in order to increase their efficiency and convenience. But, clearly nature has limits to this miniaturization process. It was Benioff [1] who proposed that the ultimate limit of miniaturization would be a quantum mechanical system. However, soon after Deutsch [3] showed that the only benefits of a quantum computer was not it's size and energy savings, a totally new way of computation was possible which made use of the entangled quantum particles and quantum parallelization. Following this, Shor [7] discovered a polynomial time prime factorization algorithm for a quantum computer which garnered a great interest into quantum computation. Together with Grover's algorithm for the fast searching of an unsorted database [6], Shor's algorithm provided the motivation for the scientists to research the possibilities of realization of a quantum computing device.

Despite the rapid progress on the theory of quantum computation, the experiments for physical implementation of a quantum computer faced major challenges. Although very basic quantum computing devices have been accomplished using different techniques, realization of a useful quantum computer in the near future seems impossible.

Because of this reason quantum computer simulation by classical computers became a popular topic. Many libraries * and standalone applications have been developed for this purpose, some of them free, † ‡ some of them proprietary §, and some languages have been developed ¶ ||. Because computation complexity increases exponentially with the number of qubits simulated, these attempts suffer great performance problems. To overcome these problems, this thesis discusses a method for quantum computer simulation on a distributed system. A similar approach has been tried by B. Oemer and Glendinning [5].

*Lib Quantum (<http://www.enyo.de/libquantum/>)

†Jaquzzi (<http://www.eng.buffalo.edu/phygons/jaQuzzi/jaQuzzi.html>)

‡Qudit (<http://www.compsoc.nuigalway.ie/damo642/QuantumSimulator/QuantumSimulator/QuantumQuditSimulator.htm>)

§Senko's simulator (<http://www.senko-corp.co.jp/qcs/qcp/qcpe.html>)

¶QCL (<http://tph.tuwien.ac.at/oemer/qcl.html>)

||Q language (<http://sra.itc.it/people/serafini/qlang/>)

2 History of Quantum Computation

In 1982, acting independently, Richard Feynman and Paul Benioff has claimed that quantum computing could be possible. Simulating quantum systems on classical computers requires an exponentially large amount of time. Feynman's first motivation was these kind of operations, he proposed that a quantum system could simulate other quantum systems more efficiently [4]. Benioff's motivation was rather continuing the trend of miniaturization in computer industry [1]. As circuits continue to shrink they will eventually reach atomic level, and quantum mechanical effects would become a major limiting factor. So the only feasible solution was to construct a computer which would make use of quantum mechanical effects.

In 1985 David Deutsch published a paper [3] which proposed a quantum Turing machine, which made use of a before unnoticed property of quantum computers. Though severely underrated at its time, Deutsch's paper would later be considered a landmark in quantum computation theory. In his paper, Deutsch showed that it is possible to perform some tasks on quantum computers much more efficiently than classical computers. Deutsch's idea was to make use of a phenomenon which is known as quantum superposition. An isolated electron, if placed in a superposition, can travel along many different paths through a circuit. If the memory could be placed in a quantum superposition before applying the algorithm, it could be in multiple states simultaneously. So just one run-through the algorithm would calculate all computational paths simultaneously. This idea (Which was to be called *quantum parallelism*) would be the foundation of all quantum algorithms.

The examples in Deutsch's paper were of limited use and due to their probabilistic nature, should be run several times to ensure the correct outcome. But these early hints in the field of quantum computation theory stirred the imaginations of many scientists, and about 10 years later Peter Shor of AT&T labs came up with a factorization algorithm for quantum computers.

In 1994 Peter Shor of AT&T developed a quantum computer algorithm for factorization [7] that is exponentially faster than any known classical algorithm. This discovery triggered a massive

interest in quantum computation, because factorization is very hard to do classically, and all best known classical algorithms are super-polynomial. Factorization is known to be in the NP (Non-Deterministic Polynomial) complexity class, but it was believed that it was not in the P (Deterministic Polynomial) class and therefore thought to be intractably difficult on classical computers. These difficulties have been the basis of many modern crypto systems - including the widely used RSA (the Rivest-Shamir-Adleman protocol), which usually rely on the difficulty of factoring a very large number, which is the product of two prime numbers (secret keys). With Shor's algorithm, it has been shown that it is possible to factorize very large numbers with quantum computers, which would shake the security of known key exchange methods.

Only two years later Lov Grover of bell labs invented a quantum algorithm for fast searching of a database [6]. Although Grover's algorithm gives only a quadratic speed up over the best possible classical algorithm, the fact that searching forms the basis of many other algorithms, and the possibility to plugged in Grover's algorithm to existing algorithms to immediately increase their efficiency, increased the importance of Grover's discovery.

These algorithms show the capabilities of a quantum computer, and the main motivation behind the research into quantum computation. It is logical to expect even more powerful algorithms to be discovered in the near future. However, an implementation of a quantum computer which would be capable of convenient applications seems far away. Currently, quantum computers can operate only on a few qubits, and under extreme environmental conditions.

3 Models of Computation

Classical computers can be modeled by a Turing machine. A *Turing Machine* is an abstract model of computation comprised of an infinitely long sequential tape, a read/write head, a set of symbols, and a set of rules for reading / writing and moving the head based on the symbol under it. These rules are called *functions*, and the symbol under the head is called the *state*. The machine moves from one state to another by executing a matching transition function. If no transition function matches, the machine halts.

The *Church-Turing Thesis* [8] says the following: A Turing machine can compute any function computable by a reasonable physical device. Programming languages can be translated into Turing machines and vice versa. Thus, the computation of any reasonable function can be expressed by a programming language.

The modern Church thesis [2] says that a probabilistic Turing machine can simulate any physical device in polynomial cost. A *probabilistic Turing Machine* can select among a set of transition functions with varying probability, instead of just one. In the following example, real numbers preceding a state indicate the probability of transitioning to that state.

$$\begin{aligned}
 |0\rangle &\rightarrow 0.5 |1\rangle + 0.5 |2\rangle \\
 |1\rangle &\rightarrow 0.2 |0\rangle + 0.8 |1\rangle \\
 |2\rangle &\rightarrow 0.4 |0\rangle + 0.3 |2\rangle + 0.3 |3\rangle \\
 |3\rangle &\rightarrow 0.3 |0\rangle + 0.7 |2\rangle
 \end{aligned}$$

Such machines can produce solutions with a negligible probability of error, which would be non computable if no error was tolerated. Probabilistic computation with negligible error is important for quantum computation.

4 Physical Background

Quantum mechanics is a branch of physics which models behavior of energy and matter at the atomic and subatomic level. Its name comes from *quanta* **.

Quantum mechanics asserts the following theories:

- (1) energy and matter exist in discrete particles,
- (2) energy and matter may behave as either particles or waves.,
- (3) movement of these particles is unpredictable, and
- (4) simultaneous measurement of two complementary values of a particle cannot be achieved.

Examples of such properties are energy, position, momentum, and angular momentum.

Last theory is called *Heisenberg's uncertainty principle*. The principle states that before measurement of a quantum particle's state, it exists in all states simultaneously. This property is called *superposition*.

Another important property is called *entanglement* or *coherence*. Pairs of particles which interact become entangled, a process called *correlation*. These particles exist in complementary states, so if one entangled particle has an "up" spin, its partner has a "down" spin. Together, entanglement and superposition enable quantum computation.

Superposition tells us that we cannot know a particle's state until it is measured. Before measurement, we can express a particle's state as a probability amplitude (defined in the next section).

The process of losing coherence is called *decoherence*, and can occur as a result of even a single stray photon. Decoherence is thought to explain why much of our world behaves according to classical, and not quantum, physics. Remaining coherent requires complete isolation of a pair from the environment, and presents one of the greatest challenges to scientists. This is why quantum computation must be reversible at every stage: thermodynamics shows that a non-reversible computation must give off heat as a byproduct, which causes interaction with the environment, which

**the discrete packets of energy proposed by Max Planck in 1900

leads to decoherence. Extensive error correcting codes are used to get around the decoherence problem in practice, but with extreme time and space overhead.

In order to explain quantum algorithms, and how they achieve the efficiency gains they are capable of, it is first necessary to understand how quantum computers work internally. The concern here is not with how quantum computers can be implemented physically, though this is an active area of research in physics. What is necessary is an understanding of how qubits work, and how they interact.

5 Mathematical Basics

Qubits are conventionally represented as Ket vectors in the *Dirac (Bra-Ket) notation*, which is common in quantum physics texts. The notation defines the Ket column vector, and its row vector transpose the Bra. The Ket vector $\vec{x}$ is denoted by $|\phi\rangle = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$, and Bra vector $\vec{y}$ is denoted $\langle\psi| = \begin{pmatrix} y_1 & y_2 \end{pmatrix}$. In quantum computation, the expression $|0\rangle$ represents quantum zero, and $|1\rangle$ represents quantum one. The state of a qubit can then be represented by the expression $\alpha|0\rangle + \beta|1\rangle$, where α and β are the amplitudes of zero and one, respectively. The probability of seeing $|0\rangle$ in this state is $|\alpha|^2$, and $|1\rangle$ is $|\beta|^2$. As long as our system is coherent, the sum of these probabilities must be 1. So, in this example, $|\alpha|^2 + |\beta|^2 = 1$. Similarly, an n-qubit register will have 2^n amplitudes, each determining the probability that the binary number corresponding to it during measurement, and the sum of their squares will be 1.

There is also an alternative, matrix notation for the quantum computation. The group of amplitudes that describe the state of a quantum register is always a unit vector. According to convention, a qubit with state $|0\rangle$, which is guaranteed to read 0 when measured, is represented by the vector

$$\begin{pmatrix} 1 \\ 0 \end{pmatrix},$$

a qubit with state $|1\rangle$ is represented by the vector

$$\begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

(these are called the basis states). So the amplitude of seeing $|0\rangle$ is given in the first row, and seeing $|1\rangle$ is given in the second row. An arbitrary qubit state is then represented by the vector

$$\begin{pmatrix} \alpha \\ \beta \end{pmatrix},$$

where α and β are complex numbers and

$$|\alpha|^2 + |\beta|^2 = 1.$$

The state of a register consisting of n qubits is represented by the tensor product of the individual states of the qubits in it.

For two matrices A and B , where A is n by m , $A \otimes B$ equals :

$$\begin{pmatrix} a_{11} & \dots & a_{1m} \\ \dots & \dots & \dots \\ a_{n1} & \dots & a_{nm} \end{pmatrix} \otimes \begin{pmatrix} a_{11}B & \dots & a_{1m}B \\ \dots & \dots & \dots \\ a_{n1}B & \dots & a_{nm}B \end{pmatrix}$$

So, the tensor product of two vectors $\vec{a}$ and $\vec{b}$ equals :

$$\begin{pmatrix} a_1 \\ \dots \\ a_n \end{pmatrix} \otimes \begin{pmatrix} b_1 \\ \dots \\ b_m \end{pmatrix} = \begin{pmatrix} a_1b_1 \\ \dots \\ a_1b_m \\ a_2b_1 \\ \dots \\ a_2b_m \\ \dots \\ \dots \\ a_nb_1 \\ \dots \\ a_nb_m \end{pmatrix}$$

So a register with two qubits where the bits are $\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$ and $\begin{pmatrix} b_1 \\ b_2 \end{pmatrix}$, respectively can be represented as

$$\begin{pmatrix} a_1 b_1 \\ a_2 b_1 \\ a_1 b_2 \\ a_2 b_2 \end{pmatrix} \text{ where, of course}$$

$$|a_1 b_1|^2 + |a_1 b_2|^2 + |a_2 b_1|^2 + |a_2 b_2|^2 = 1.$$

A quantum program which transforms a register of n qubits from an input state to an output state can be represented as 2^n by 2^n matrix, which, when multiplied by the input state vector, gives the output state vector. Because Quantum Computation should be information lossless (ie reversible), this matrix should be a unitary matrix. The combined effect of several single-qubit programs which run parallelly on the qubits in a register can be represented as the tensor product of their individual matrices.

6 Quantum Computation

We are concerned with using quantum computation to produce solutions to problems which are considered intractable for classical computers. ^{††} To see how quantum computers can compute classically intractable problems, we need to examine how they perform computation.

The building block of the quantum computer is the *qubit*, or quantum bit. A qubit is an abstract value, represented by nano scale properties of physical matter. Qubits have been implemented using ion charge, photon polarization, nuclear spin of atoms, and nuclear spin of impurities in silicon chips.

Classical bits abstractly represent values of 0 or 1 by the absence or presence of voltage. In contrast, a qubit represents both a 0 and 1 simultaneously due to superposition.

Since qubits represent two values at once, n qubits represent 2^n values. Thus, quantum computers operate on exponential data with only linear input. For example: two qubits represent the values 00 01 10 and 11 simultaneously. The probability of the value which will be observed is represented by a complex number called *probability amplitude*.

Probability amplitudes are mathematically analogous to a wave: they can be positive or negative, and are subject to interference. The probability amplitude of a value is the sum of probability amplitudes over all entangled particles. The classical probability of a value is given by the square of its probability amplitude. Quantum computation is performed by manipulating probability amplitudes of qubits.

A qubit can be represented by unit vector in the Hilbert space C^2 , where C is the set of complex numbers. An n -qubit system can be described by the tensor product of these Hilbert spaces:

We can compute a function f on these qubits by applying a unitary operator U :

when f is one-to-one, ie reversible, we can always find a unitary operator. Any classical function we wish to compute can be made reversible using extra bits and acceptable operational over-

^{††}An *intractable* problem is one for which execution time grows exponentially as input size increases linearly.

head. We can think of U as a basic quantum gate, performing the role of a classical truth table. We can represent U as a unitary matrix, which by definition is reversible.

To apply U and thereby compute our function f , we find the Hamiltonian H which solves the equation :

Absolution exists as long as U is unitary. By applying U to a vector on n qubits we apply f to all possible 2^n inputs in parallel. This parallelism is how quantum computation overcomes classically intractable problems.

Obtaining the computed solution presents a new challenge. The superposition of n qubits have 2^n possible states, and is called a *wave function*. By observing the value of a vector of qubits we *collapse* this wave function, selecting one state from exponentially many to produce a classical bit vector. This collapsing effectively destroys the solution. However, we can use the interference property of probability of observing the correct answer.

Quantum computing is not without drawbacks. Firstly, the exponential parallelism inherent in quantum computation does not imply exponential computational power. In addition, practical implementation is hindered by decoherence, caused by interference from the environment as well as the exponential decay times of quantum states. Quantum error correcting codes are necessary to mitigate decoherence and error propagation in the real world. These codes require many physical qubits to accurately encode a single logical qubit of information. Finally, certain quantum architectures have impractical requirements, such as operating at near absolute zero temperature. Current quantum computers require classical control components, and CMOS transistors cannot be used in such an environment.

Due to these difficulties, simulation is important for the development and advancement of quantum algorithms. Quantum simulators run on classical computers and allow state observation, analysis of error accumulation, and feasibility studies. However, simulating a 2^n dimensional Hilbert space on a classical system requires manipulating bit vectors of size 2^n , whereas quantum com-



Figure 1: Dr. Isaac Chuang, research staff member at IBM's Almaden Research Center (San Jose, Calif.), holds a quantum computer – a glass tube containing specially designed molecules that can solve some of the most difficult mathematical problems exponentially faster than a conventional computer.

puters need only n qubits. As a result, quantum simulators are restricted to small numbers of qubits.

Real quantum computers do exist which operate on 5-7 qubits. Shor's algorithm was successfully demonstrated on a 7-qubit machine.

7 Simulation

This project aims to produce quantum computer simulator that will run on classical computers. Unfortunately, the simulator will be inefficient as it is simulating quantum states on a classical system. Although a quantum computer works on different physical paradigm to a classical computer, quantum computers cannot do anything that classical computers cannot. Classical computers can simulate quantum computers to arbitrary precision by storing vector representations of the quantum states and perform any required rotation upon them. The reason for using quantum computers is merely the increase in efficiency possible over classical computational devices. However, using a classical system to simulate a quantum computer will nullify any efficiency gains due to the extra processing required. It would in fact be much less efficient, as in order to simulate a quantum computer a great deal of information that is implicit in quantum states must be explicitly programmed and manipulated in the classical system. In order to simulate even a small quantum algorithm, consisting of 100 qubits, would require 2^{100} (approximately 10^{30}) complex numbers to represent the quantum state. No existing digital computer, can handle numbers of this magnitude, and that performing any kind of rotation of a vector in a space with a dimension of 10^{30} is far beyond the computational capacity of any foreseeable classical computer.

So though a quantum computer can be simulated by a classical computer, the simulation becomes increasingly inefficient as the number of qubits in the simulation increases. Computationally quantum mechanics is very difficult, as the size of the matrices required to store the state of the system grows exponentially with the number of qubits.

8 Parallel Simulation

In this thesis, the LAM message passing interface (MPI) has been used to distribute the computation (maintained by Indiana university)^{‡‡}. The program is SIMD (Single Instruction Multiple Data), with each node running a copy of the same code, but operating on different data. The heaviest computational burden in the simulation of a quantum computer is the matrix multiplication when a register is applied a quantum program. The resulting rows of the output vector contain n multiplications, and $n-1$ additions for a quantum register with n qubits. Although all nodes will be running the same code, one of them will perform special tasks, it will be used to distribute the data (input vector and quantum program) and get the results (output vector). This node will be called the master node. A strategy is to keep the probability amplitudes of each qubit on different processors, and apply the quantum gates to individual qubits. However this approach doesn't let us obtain the output vector after the execution of the quantum program, so it is required to collect the qubit's values in a single processor and perform a tensor product to get the output vector, which implies serious communication and computation overheads. Another strategy is to assign each row of the input vector to individual nodes, and perform the computation of the corresponding row of the outcome vector on this node. Since only one row of the Quantum program is needed to calculate the output vector's row, the quantum program can be efficiently distributed. However, with this method, the nodes have to communicate the members of the input vector with each other, which increases the communication overhead, and code complexity. Since all the nodes must have all the members of the input vector to calculate the corresponding row of the output vector, it is quite logical to distribute a copy of the input vector to all processors before execution. The program supplied with this thesis uses this strategy, it broadcasts the input vector to all nodes, to minimize the communication overhead. Then the quantum program is divided into chunks of appropriate sizes and distributed to corresponding nodes. With this strategy, it is possible to efficiently dis-

^{‡‡}<http://www.lam-mpi.org/>

tribute the bulk of the computation among the worker nodes. After the computation is finished, the nodes begin broadcasting their results. Actually, if each node sent its result to the master node, it would be possible to get the output vector on the master node. However, if the quantum programs are applied sequentially, this approach needs to send the output vector to the worker nodes as the input vector of the next quantum program, thus increasing the communication overhead. So it is wise to keep a copy of the output vector on all nodes to minimize the communication overhead. Depending on the network architecture, it may be more efficient to use a hypercube template to gather all the data on all the nodes (if nodes are connected with a hypercube topology). However, with a classical star topology, plain broadcasts of the results will be more efficient.

9 Library Documentation

Structure of a Matrix :

A matrix struct contains column and row sizes and a pointer to an array of arrays of complex data structs. The GCC extension for complex numbers have been used for this project. The matrix struct also holds the row and column sizes of the matrix.

```
typedef struct {
int row_size;
int column_size;
__complex__ double ** value;
} matrix;
```

How to create a matrix :

with the create matrix function. column size and row size are passed as arguments. ex :

```
matrix * m = create_matrix (2,2);
```

How to initialize a matrix :

A freshly created matrix's elements are not initialized. the init matrix function zeroes all elements of a matrix. ex :

```
init_matrix(m);
```

or you can set the elements to a fixed value with the

```
init_matrix_c
```

(init matrix to complex value) function.

```
__complex__ double a = 4 + 3i;
init_matrix_c(m,a);
```

`init_matrix_n`

function sets the real parts of the elements to the given value.

`init_matrix_n(m, 3.17);`

or you can set the matrix to a predefined matrix, currently, hadamard gate (for a 2 by 2 matrix) and CNOT gate (for a 4 by 4 matrix) are available

`init_hadamard(m);`

`init_CNOT(m);`

how to set matrix values by hand :

you can reach the elements in a matrix with the 'value' pointer in the matrix. You have to provide row and column numbers.

`matrix * m2 = create_matrix(4,1);`

`m2->value[0][0]=1;`

`m2->value[1][0]=2;`

`m2->value[2][0]=3;`

`m2->value[3][0]=4;`

this matrix is a vector containing :

$$\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

how to print a matrix :

simply, with the

`print_matrix`

function.

```
print_matrix (m2);
```

How to create a quantum register :

a quantum register with n qubits is a vector with 2^n dimensions. When creating a quantum register with

```
create_qubit
```

function, you provide the size of the register with the first parameter, and then provide the value (an integer value) which the register will hold as its initial value. ex :

```
m = create_qubit (2,3);
```

will create a 1x4 matrix with 0 0 0 1

as it's initial value. 3 is represented as 11 in binary, so both qubits should keep 1.

```
m = create_qubit (2,0);
```

will create 1 0 0 0

providing a larger value than the register can hold will result in an error.

Deconstructor

matrix() function will destroy a matrix, freeing all the memory allocated.

Matrix Operations

tensor product : returns a pointer to a matrix which is the tensor product of two given matrices.

The original matrices are kept untouched. Ex:

```
matrix * m3 ;
```

```
m3 = tensor_product(m1,m2);
```

Matrix multiplication : returns a pointer to a matrix which is the multiplication of two given matrices. The original matrices are kept untouched. Ex:

```
matrix * m3 ;
m3 = dot_product(m1,m2);
```

Matrix Transmission functions : There are three functions for easy transmission of matrices within the lam-mpi environment. Sending a matrix :

`send_matrix`

sends a matrix to a specified node. Ex:

```
send_matrix(m2,2);
```

Receiving a matrix :

`get_matrix`

gets a matrix from the specified node. Ex:

```
matrix * m = get_matrix(0);
```

Broadcasting a matrix : broadcasts a matrix. The first argument is a pointer to the matrix to be sent (this can be (should be) a null pointer for the nodes who will receive the matrix, and a pointer to an actual matrix for the node who will send the matrix) the second argument is the rank of the node, and the last argument is the rank of the node who will make the broadcast. Ex:

```
matrix * m2;
if(rank == 0)
{
m2 = create_matrix (4,4);
```



```
__complex__ double a = 4 + 3i;  
init_matrix_c(m2,a);  
}  
m2 = broadcast_matrix(m2,rank,0);  
printf("rank %d received matrix :\n",rank);  
print_matrix(m2);
```

10 Conclusion

As long as Quantum computers with a sufficient memory to apply some useful functions for some basic tasks are still under development, Quantum Simulators will provide the possibility for scientists to research new algorithms. The difficulties encountered in simulating Quantum Computers with Classical Computers make us develop new methods. Parallel Computation is one of the most promising ways to simulate a Quantum Computer and recently many researchers are interested in this subject.

In this thesis, the basics of Quantum Computation and some strategies for a parallel simulation have been discussed, and one of them has been implemented. Due to time limitations, the library could not be developed fully, many functions to provide basic Quantum Computer operations (ie Quantum Fourier Transform) was unfinished. A complete library could prove quite useful for research into Quantum Computation algorithms.

References

- [1] Paul Benioff. The computer as a physical system : A microscopic quantum mechanical hamiltonian model of computers as represented by turing machines. *J. Statist. Phys.*, 22:563–591, 1980.
- [2] J. Church, 1936.
- [3] David Deutsch. Quantum theory, the church-turing principle and the universal quantum computer. *Proceedings of the Royal Society of London*, A400:97–117, 1985.
- [4] Richard P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21, 1982.
- [5] Ian Glendinning and Bernhard Oemer. Parallelization of the general single qubit gate and cnot for the qc-lib quantum computer simulator library, 2003.
- [6] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proc. 28th Annual ACM Symposium on Theory of Computing*, ACM, New York, 1996.
- [7] Peter W. Shor. Polynomial-time algorithms for primefactorization and discrete logarithms on a quantum computer. *Proceedings of the 35th Annual Symposium on Foundations of Computer Science, Santa Fee, IEEE Computer Society Press*, pages 124–134, 1994.
- [8] A. M. Turing, 1936.